OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

Analysis of Process Management and Deadlock-Handling Techniques

Submitted by

MOHAMMED FAREETH S A

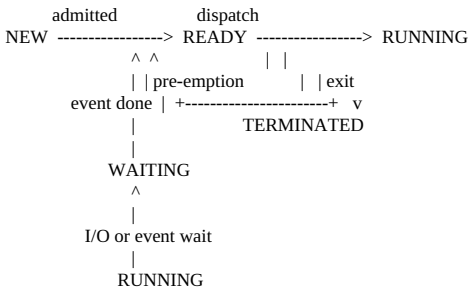
Register Number: 192511288

Course: CSA0417 – Operating Systems

Q1 – Processes and Process States

1.1 Process-state model

A process moves between five main states as it executes. The operating system records the current state and the information needed to pause and resume the process.



→ **New:** the process is being created and its PCB is initialized.

→ **Ready:** the process is in memory and waiting for CPU time.

→ **Running:** its instructions are currently executing on a CPU core.

→ **Waiting/Blocked:** it cannot continue until an event completes, such as disk, network or camera I/O.

→ **Terminated:** execution has ended; the OS releases resources after the exit status is collected.

1.2 Process Control Block (PCB)

PCB field	**Purpose**
PID and parent PID	Uniquely identifies the process and its parent.
Process state	New, ready, running, waiting or terminated.
Program counter	Address of the next instruction to execute.
CPU registers	Saved register values used when the process is switched out.
Scheduling information	Priority, queue links, CPU time and scheduling parameters.
Memory information	Page tables, address-space details, code/data/stack regions.
I/O and file information	Open file descriptors, devices and pending I/O requests.
Accounting and security	User/group IDs, resource usage, permissions and limits.

1.3 Fork program and observation

The following program creates child processes, prints their PIDs and briefly keeps them alive so that ps and /proc can inspect them.

```
/* q1_processes.c */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
#include <sys/wait.h>

int main(void)
{
    const int children = 4;
    pid_t pids[children];

    for (int i = 0; i < children; i++) {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); return 1; }
        if (pid == 0) {
            printf("child %d: PID=%ld PPID=%ld\n",
                   i, (long)getpid(), (long)getppid());
            fflush(stdout);
            sleep(15);
            _exit(0);
        }
        pids[i] = pid;
    }

    printf("parent PID=%ld; inspect with ps and /proc\n", (long)getpid());
    fflush(stdout);
    for (int i = 0; i < children; i++) waitpid(pids[i], NULL, 0);
    return 0;
}
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra q1_processes.c -o q1_processes
./q1_processes
```

In another terminal:

```
ps -o pid,ppid,stat,comm -C q1_processes
ps -ef | grep q1_processes
cat /proc/<PID>/status
cat /proc/<PID>/stat
```

1.4 Evaluation for 2,000 concurrent sessions

The delay is not caused by process count alone. CPU scheduling, memory pressure, lock contention, network waits, database latency and video encoding compete for resources. Many sessions spend substantial time waiting for I/O, so keeping every session as an independent active process adds scheduling and memory overhead without necessarily improving throughput.

Resource factor	Illustrative estimate for 2,000 processes	Evaluation
Context switching	About 2–10 µs per switch on a modern server, excluding cache disruption and scheduler overhead	Thousands of runnable tasks can cause frequent switches and cache misses.
Per-process memory overhead	Roughly 1–4 MB resident overhead per lightweight service process; application memory is additional	About 2–8 GB for overhead alone at 2,000 processes; actual values vary greatly.
I/O waiting	Network/database waits can range from sub-ms to tens of ms or more	A large blocked population is normal, but thread/process count should be bounded.
Operational cost	2,000 independent address spaces and descriptors	More bookkeeping, monitoring, resource limits and failure-management work.

1.5 Design decision and alternative

Recommendation: use a fixed pool of worker processes for CPU-heavy or security-sensitive work, combined with asynchronous I/O or a bounded thread pool for session handling. Avoid blindly creating one process per student. A worker pool controls concurrency, limits memory growth and provides process-level isolation between groups of jobs.

→ One process per student: strongest address-space isolation, but highest process and memory overhead.

→ One thread per student: lower creation and switching overhead; threads share memory and can communicate cheaply, but one memory-corruption bug can affect the entire server.

→ Fixed worker pool: bounded number of workers, predictable resource use and good throughput; jobs wait in a queue when all workers are busy.

A POSIX thread commonly has a configurable stack; Linux defaults can reserve several MB per thread, but a server can configure smaller safe stacks, for example 128–512 KB where appropriate. A few KB is not a safe universal assumption. If 2,000 threads each reserve 256 KB, stack reservation is about 500 MB, before application data and runtime structures.

Model	Creation/dispatch cost	Memory and isolation
Process per session	Higher	Separate virtual address spaces; stronger fault isolation.
Thread per session	Usually lower	Shared address space; each thread needs stack and metadata.
Fixed worker pool	Amortized; workers reused	Bounded memory and concurrency; queueing under overload.

Complexity: creating N processes or threads is O(N) total; storing session descriptors is O(N). A pool of W workers has O(W) execution contexts while the waiting queue can contain O(N) session requests. A worker crash affects its current job; a process-per-session crash is usually isolated to one session, while a thread crash caused by memory corruption can terminate the whole server.

Q2 – CPU Scheduling

2.1 Job set and definitions

Process	Arrival time	Burst time	Priority
P1	0	8	2
P2	1	4	1
P3	2	9	4
P4	3	5	3

Turnaround time (TAT) = completion time − arrival time. Waiting time (WT) = TAT − burst time. Response time = first CPU start − arrival time.

2.2 FCFS – First Come, First Served

Gantt chart: 0 | P1 | 8 | P2 | 12 | P3 | 21 | P4 | 26

Process	Completion	TAT	WT	Response
P1	8	8	0	0
P2	12	11	7	7
P3	21	19	10	10
P4	26	23	18	18
Average		15.25	8.75	8.75

FCFS is simple and fair by arrival order, but suffers from the convoy effect: short thumbnail jobs can wait behind a long film.

2.3 SJF – Shortest Job First (non-preemptive)

At time 0 only P1 is available, so it runs first. At time 8, P2, P3 and P4 are ready; P2 is shortest, followed by P4 and P3.

Gantt chart: 0 | P1 | 8 | P2 | 12 | P4 | 17 | P3 | 26

Process	Completion	TAT	WT	Response
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9
Average		14.25	7.75	7.75

SJF can reduce average waiting time when burst estimates are accurate. Long jobs can starve if short jobs continue arriving.

2.4 SRTF – Shortest Remaining Time First (preemptive)

A newly arrived job pre-empts the running job if its burst is shorter than the current job’s remaining time.

Gantt chart: 0 | P1 | 1 | P2 | 5 | P4 | 10 | P1 | 17 | P3 | 26

Process	Completion	TAT	WT	Response
P1	17	17	9	0
P2	5	4	0	0
P3	26	24	15	15
P4	10	7	2	2

Process	Completion	TAT	WT	Response
Average		13.00	6.50	4.25

SRTF improves short-job response but requires remaining-time estimates and more scheduling decisions. A continuous stream of short arrivals can starve a long job.

2.5 Priority scheduling (non-preemptive)

Assumption: smaller number means higher priority. Priorities are P1=2, P2=1, P3=4 and P4=3. P1 starts because it is the only job at time 0. Once it finishes, the ready job with the highest priority is selected.

Gantt chart: 0 | P1 | 8 | P2 | 12 | P4 | 17 | P3 | 26

Process	Completion	TAT	WT	Response
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9
Average		14.25	7.75	7.75

Priority scheduling can starve low-priority jobs. Ageing gradually increases the priority of a process the longer it waits, preventing indefinite postponement.

2.6 Round Robin and quantum study

Round Robin gives each ready process a time slice (quantum). With quantum = 3, the Gantt chart and metrics are:

Gantt chart: 0 | P1 | 3 | P2 | 6 | P3 | 9 | P4 | 12 | P1 | 15 | P2 | 16 | P3 | 19 | P4 | 21 | P1 | 23 | P3 | 26

Process	Completion	TAT	WT	Response
P1	23	23	15	0
P2	16	15	11	2
P3	26	24	15	4
P4	21	18	13	6
Average		20.00	13.50	3.00

Quantum	Average TAT	Average WT	Approx. CPU dispatches / slices	Comment
1	19.00	12.50	26	Responsive, but many context switches.
3	20.00	13.50	10	Balanced compromise for this small set.
5	17.50	11.00	6	Fewer switches; short jobs still get service.
20	15.25	8.75	4	Behaves like FCFS for this workload; poor initial responsiveness.

A small quantum improves responsiveness and fairness but increases context-switch overhead. A very large quantum reduces switching but loses the interactive advantage of Round Robin.

2.7 Completed Round Robin C program

```
#include <stdio.h>
#define N 4
int main(void) {
    const char *name[N] = {"P1","P2","P3","P4"};
    int arrival[N] = {0,1,2,3}, burst[N] = {8,4,9,5}, quantum = 3;
    int left[N], finish[N], first[N], queue[128], head=0, tail=0;
```

```
    int added[N]={0}, time=0, done=0;
    for(int i=0;i<N;i++){ left[i]=burst[i]; first[i]=-1; }
    queue[tail++]=0; added[0]=1;
    printf("Gantt:");
    while(done<N) {
        int p=queue[head++];
        if(first[p]<0) first[p]=time;
        int run = left[p] < quantum ? left[p] : quantum;
        int old=time; time += run; left[p] -= run;
        printf(" %s(%d-%d)", name[p], old, time);
        for(int i=0;i<N;i++)
            if(!added[i] && arrival[i]<=time) {
                queue[tail++]=i; added[i]=1;
            }
        if(left[p]>0) queue[tail++]=p;
        else { finish[p]=time; done++; }
    }
    double tw=0,tt=0;
    printf("\\nProcess TAT WT\\n");
    for(int i=0;i<N;i++) {
        int tat=finish[i]-arrival[i], wt=tat-burst[i];
        printf("%-7s %4d %3d\\n",name[i],tat,wt);
        tt+=tat; tw+=wt;
    }
    printf("Average TAT = %.2f, average WT = %.2f\\n",tt/N,tw/N);
    return 0;
}
```

Compile: **gcc -std=c11 -Wall -Wextra rr.c -o rr**; run: **./rr**. The expected output for quantum 3 is the Gantt chart and average values shown above.

2.8 Multilevel Feedback Queue (MLFQ) and recommendation

An MLFQ starts new jobs in a high-priority queue with a short quantum. Jobs that repeatedly use their entire quantum are moved to lower-priority queues with longer quanta. Interactive thumbnail jobs tend to finish quickly near the top; long film-encoding jobs move down and receive larger CPU slices. Periodic priority boosts prevent starvation.

Recommendation: use an MLFQ or a priority-aware fair scheduler for the video service, with reserved capacity for short interactive jobs and background capacity for long encodes. For the supplied example, SRTF gives the lowest average waiting time (6.50 units) and best average response (4.25), but requires reliable burst estimates and can starve long jobs. MLFQ offers a practical compromise under mixed, changing workloads.

Q3 – Threads)

3.1 Shared and private thread resources

Shared by threads in one process	**Private to each thread**
Code/text segment	Thread ID and register context
Global and heap data	Stack and local automatic variables
Open file descriptors	Program counter and CPU registers
Address space and mapped memory	Thread-local storage
Process credentials and many process resources	Scheduling state and signal mask

3.2 Thread implementation models

→ **User-level threads:** managed by a runtime library; switching can be fast, but a blocking system call may block the whole process in a many-to-one design.

→ **Kernel-level threads:** known and scheduled by the OS; can run in parallel on multiple cores, but creation and switching require kernel support.

→ **Many-to-one:** many user threads map to one kernel thread; low overhead but no true multicore parallelism.

→ **One-to-one:** each user thread maps to a kernel thread; supports parallelism but has higher kernel resource overhead.

→ **Many-to-many:** many user threads are multiplexed over a pool of kernel threads; balances flexibility and parallelism but is more complex.

3.3 POSIX thread pool program

The following program simulates processing 64 camera frames. Each frame performs a deterministic CPU workload. The same source can be run with 1, 4, 8 and 16 worker threads. A fixed pool avoids creating 64 long-lived threads for a 64-camera input set.

```
/* q3_threads.c */
#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>

#define CAMERAS 64
#define WORK 1200000
static int next_camera = 0;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static volatile uint64_t checksum = 0;

static void *worker(void *arg) {
    (void)arg;
    for (;;) {
        pthread_mutex_lock(&m);
        int id = next_camera++;
        pthread_mutex_unlock(&m);
        if (id >= CAMERAS) break;

        uint64_t x = (uint64_t)id + 1;
        for (int k=0; k<WORK; k++)
            x = (x * 1664525u + 1013904223u) & 0xffffffffu;

        __sync_fetch_and_add(&checksum, x);
    }
    return NULL;
}
int main(int argc, char **argv) {
    int n = argc > 1 ? atoi(argv[1]) : 4;
    if(n<1 || n>64) return 1;
    pthread_t *t = calloc((size_t)n, sizeof *t);
    struct timespec a,b;
    clock_gettime(CLOCK_MONOTONIC,&a);
    for(int i=0;i<n;i++) pthread_create(&t[i],NULL,worker,NULL);
    for(int i=0;i<n;i++) pthread_join(t[i],NULL);
    clock_gettime(CLOCK_MONOTONIC,&b);
    double sec=(b.tv_sec-a.tv_sec)+(b.tv_nsec-a.tv_nsec)/1e9;
    printf("threads=%d checksum=%llu time=%.4f s\\n",
           n,(unsigned long long)checksum,sec);
```

```
    free(t); pthread_mutex_destroy(&m);
    return 0;
}
```

Compile and run:

```
gcc -O2 -std=c11 -Wall -Wextra -pthread q3_threads.c -o q3_threads
./q3_threads 1
./q3_threads 4
./q3_threads 8
./q3_threads 16
```

3.4 Illustrative speed-up results

The following are plausible example values for a machine with multiple CPU cores, not measured results. Sequential time is taken as 12.8 seconds for illustration. Speed-up = sequential time / parallel time.

Workers	**Example elapsed time**	**Speed-up**	**Efficiency**
1	12.8 s	1.00×	100%
4	3.9 s	3.28×	82%
8	2.3 s	5.57×	70%
16	1.8 s	7.11×	44%
64 (one per camera)	2.0 s	6.40×	10%

Speed-up stops increasing because of serial work, mutex contention, memory bandwidth, cache misses, CPU-core limits and scheduling overhead. Creating 64 threads may cost more than it saves for small frames.

3.5 Race condition and protection

If two threads increment a shared counter using counter++, both can read the same old value and write back the same incremented value. Some updates are lost. Protect the update with a mutex or use a C11 atomic variable:

```
pthread_mutex_lock(&counter_lock);\ncounter++;\npthread_mutex_unlock(&counter_lock);\n/* Or: atomic_fetch_add(&counter, 1); */
```

3.6 Alternative: camera processes

Using processes provides separate address spaces and stronger crash isolation. Frames and results must be transferred through pipes, message queues, sockets or shared memory. Process creation and context switching usually cost more than thread operations; shared memory can reduce copying but requires synchronization. Threads are recommended for trusted frame-processing tasks in one service. Separate processes are preferable when camera decoders are untrusted or a crash must not affect other decoders.

Complexity: processing C frames with total pixel work W is O(W) regardless of thread count. With p workers, ideal compute time approaches O(W/p), but synchronization and serial overhead limit speed-up. Thread bookkeeping is O(p); per-process address-space and IPC overhead is typically larger.

Q4 – Inter-Process Communication

4.1 Pipe-based sensor-to-analyser communication

A pipe is a byte stream between processes. The parent writes sensor readings; the child reads them. When every write-end descriptor is closed, read() returns 0, which indicates end-of-file. The parent must close its copy of the write end or the child can wait forever for more data.

```
/* q4_pipe.c */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

int main(void) {
    int fd[2];
    if(pipe(fd)==-1){perror("pipe");return 1;}
    pid_t pid=fork();
    if(pid<0){perror("fork");return 1;}
    if(pid==0) {
        close(fd[1]);
        int r, count=0, sum=0, max=0;
        ssize_t n;
        while((n=read(fd[0],&r,sizeof r))>0) {
            count++; sum+=r; if(r>max) max=r;
        }
        if(n<0) perror("read");
        printf("analyser: %d readings, average %.1f, max %d\\n",
               count,count?(double)sum/count:0.0,max);
        close(fd[0]); fflush(stdout); _exit(0);
    }
    close(fd[0]);
    int readings[]={31,33,36,34,41,38};
    for(int i=0;i<6;i++)
        if(write(fd[1],&readings[i],sizeof readings[i])<0) perror("write");
    close(fd[1]);
    waitpid(pid,NULL,0);
    printf("reader: done\\n");
    return 0;
}
```

Compile and run: **gcc -std=c11 -Wall -Wextra q4_pipe.c -o q4_pipe**, then **./q4_pipe**.

Expected output:

```
analyser: 6 readings, average 35.5, max 41\nreader: done
```

If close(fd[1]) is removed from the parent, the child does not receive EOF after the six readings because a write-end descriptor remains open. The child blocks waiting for more bytes, while the parent waits for the child: the program appears hung.

4.2 Shared-memory alternative

Shared memory maps the same physical memory region into both processes. The producer writes readings into a shared ring buffer and the consumer reads them. Unlike a pipe, data need not be copied through the kernel pipe buffer for every transfer. However, the two processes must coordinate access with process-shared semaphores or another synchronization mechanism.

```
/* Shared-memory design outline (POSIX) */
shm_open("/weather", O_CREAT|O_RDWR, 0600);
ftruncate(fd, sizeof(struct shared_buffer));
ptr = mmap(NULL, sizeof(struct shared_buffer),
        PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);

/* shared structure contains data[], in, out, sem_empty, sem_full */
sem_wait(&sem_empty);       /* producer waits for free slot */
lock_shared_mutex();
data[in] = reading;
in = (in + 1) % CAPACITY;
unlock_shared_mutex();
sem_post(&sem_full);

/* consumer performs sem_wait(sem_full), reads one item,
   advances out, then sem_post(sem_empty). */
munmap(ptr, sizeof(struct shared_buffer));
shm_unlink("/weather");
```

4.3 Comparison and approximate transfer timings

Method	Copying and synchronization	1 MB	10 MB	100 MB
Pipe	Kernel pipe buffer; producer and consumer block/wake through kernel	~0.7 ms	~6.8 ms	~68 ms
Shared memory	Shared mapping avoids per-message pipe-buffer copy; requires semaphores/mutex	~0.3 ms	~3.0 ms	~30 ms
Message queue	Kernel-managed messages; convenient message boundaries, copying and queue overhead	~1.0 ms	~9 ms	~90 ms

Complexity: transferring n bytes is O(n) for every method. Pipe and message-queue overhead includes kernel copying and per-operation system calls; shared-memory throughput is limited by memory bandwidth and synchronization. Shared-memory setup is O(1) per mapping, but the ring-buffer bookkeeping is O(1) per item.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors

5.1 Critical-section requirements

→ **Mutual exclusion:** at most one process/thread executes the protected critical section at a time.

→ **Progress:** if no thread is in the critical section and some want to enter, selection of the next entrant cannot be postponed indefinitely.

→ **Bounded waiting:** after a thread requests entry, there is a finite bound on how many times others may enter first.

5.2 Peterson's solution

For two processes, Peterson's algorithm uses flag[2] to indicate interest and turn to break ties. Each process sets its flag, gives the other process priority, and waits while the other is interested and it is the other process's turn. Under the classic sequentially consistent memory model, this satisfies mutual exclusion, progress and bounded waiting for two participants.

```
/* Conceptual Peterson algorithm for process i; j = 1-i */
flag[i] = true;
turn = j;
while (flag[j] && turn == j) { /* wait */ }
/* critical section */
flag[i] = false;
```

On modern processors, compiler and CPU reordering and weak memory models can invalidate the assumptions unless appropriate atomic operations and memory ordering are used. Peterson's algorithm is mainly educational; production code should use pthread mutexes or C11 atomics with a proven protocol.

5.3 Binary and counting semaphores

A binary semaphore has values 0 or 1 and can protect a single critical section. A counting semaphore represents multiple identical available resources. wait() decrements the count or blocks if no resource is available; signal() increments it and may wake a waiting thread.

```
sem_wait(&seat_lock);\nif (available_seats > 0) available_seats--;\nsem_post(&seat_lock);
```

A mutex is usually preferable for protecting a short in-memory critical section because it has ownership semantics. A counting semaphore is useful for tracking a number of available seats, buffer slots or connections.

5.4 Bounded-buffer producer–consumer solution

The empty semaphore counts free buffer slots; full counts occupied slots; the mutex protects the ring-buffer indexes and contents. The code below completes both TODO sections.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define SIZE 5
#define ITEMS 20
int buffer[SIZE], in=0, out=0;
sem_t empty, full;
pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER;
long consumed_sum=0;

void *producer(void *arg) {
   (void)arg;
   for(int item=1; item<=ITEMS; item++) {
      sem_wait(&empty);
      pthread_mutex_lock(&lock);
      buffer[in]=item;
      in=(in+1)%SIZE;
      pthread_mutex_unlock(&lock);
      sem_post(&full);
   }
   return NULL;
}
void *consumer(void *arg) {
   (void)arg;
   for(int k=0;k<ITEMS;k++) {
      sem_wait(&full);
      pthread_mutex_lock(&lock);
      int item=buffer[out];
      out=(out+1)%SIZE;
```

```
        pthread_mutex_unlock(&lock);
        sem_post(&empty);
        __sync_fetch_and_add(&consumed_sum,item);
    }
    return NULL;
}
int main(void) {
    sem_init(&empty,0,SIZE); sem_init(&full,0,0);
    pthread_t p,c;
    pthread_create(&p,NULL,producer,NULL);
    pthread_create(&c,NULL,consumer,NULL);
    pthread_join(p,NULL); pthread_join(c,NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\\n",
        ITEMS,consumed_sum,ITEMS*(ITEMS+1)/2);
    sem_destroy(&empty); sem_destroy(&full);
    return 0;
}
```

Compile: **gcc -std=c11 -Wall -Wextra -pthread q5_buffer.c -o q5_buffer**. Run 100 times with a shell loop:

```
for i in $(seq 1 100); do ./q5_buffer; done
```

Expected every run: **items consumed: 20, sum = 210 (expected 210)**. For multiple producers and consumers, each producer should receive a unique item range or use an atomic item allocator. The same empty/full/mutex pattern preserves each buffer operation. Removing the mutex can cause concurrent index updates and buffer corruption; with one producer and one consumer the semaphore ordering may mask some races, but it is not a sound multi-producer/multi-consumer design.

5.5 Readers–writers and dining philosophers

Readers–writers: multiple readers may access shared data concurrently, but writers require exclusive access. A mutex protects the reader count; the first reader locks the data resource and the last reader releases it. A fair read-write lock or turnstile can prevent writer starvation.

```
reader: lock(count_mutex); readers++; if(readers==1) wait(resource); unlock(count_mutex);\n/* read */\nlock(count_mutex); readers--; if(readers==0) signal(resource); unlock(count_mutex);\
```

Dining philosophers: deadlock can be avoided by imposing a global order on fork/resource acquisition, allowing at most N−1 philosophers to compete, or using a monitor that grants both resources atomically. A global resource order prevents circular wait.

5.6 Monitors and condition variables

A monitor encapsulates shared data and the operations that access it. Only one thread executes a monitor procedure at a time. Condition variables allow a thread to wait until a predicate becomes true; the predicate must always be checked in a while loop because wake-ups do not guarantee that the condition still holds.

```
pthread_mutex_lock(&m);\nwhile (count == CAPACITY) pthread_cond_wait(&not_full, &m);\n/* insert item */\npthread_cond_signal(&not_empty);\npthread_mutex_unlock(&m);
```

5.7 Booking-system locking decision

Locking strategy	Illustrative throughput	Advantages and limitations
One global lock	~8,000 bookings/s	Simple correctness; all trains serialize, causing a bottleneck.
One lock per train	~25,000 bookings/s	Different trains proceed concurrently; a booking within one train remains serialized.
One lock per coach/seat group	~42,000 bookings/s	More parallelism, but cross-coach transactions and seat selection need careful coordination.

The throughput figures are illustrative estimates, not measured results. **Recommendation:** use a lock per train as the safe baseline, then refine to coach/seat-level locking only after transaction rules and deadlock-free lock ordering are proven. A database uniqueness constraint on (train_id, seat_id, journey_id) should be the final protection against duplicate sales.

5.8 Atomic operations and spin locks

A C11 atomic counter can use atomic_fetch_add() for a simple counter update without a mutex. It is suitable for a short independent update, but it cannot by itself protect a multi-step seat-allocation transaction. A spin lock repeatedly checks until the lock becomes free; it wastes CPU while waiting and is unsuitable for long critical sections or oversubscribed systems.

Threads	Atomic update estimate	Spin-lock CPU behaviour
1	~5–20 ns/update	No contention; short acquisition.
4	~20–100 ns/update	Cache-line bouncing under contention.
16	~100–500 ns/update or more	High contention; many threads may burn CPU while spinning.

These are rough order-of-magnitude examples and vary by architecture. For a long critical section, a blocking mutex/semaphore is better because waiting threads sleep rather than continuously consume CPU. For a short, low-contention counter update, atomics are usually preferable.

Q6 – Deadlock Handling and the Banker’s Algoriths)

6A. Characterise deadlock

Deadlock is a condition in which a set of processes is permanently blocked because each process is waiting for a resource held by another process in the set. Four conditions must hold simultaneously:

→ **Mutual exclusion:** at least one resource is non-shareable, such as an exclusive GPU context or a single floating licence.

→ **Hold and wait:** a job holds one resource while requesting another, such as holding a GPU while waiting for storage.

→ **No pre-emption:** a resource cannot be forcibly taken away; it is released voluntarily.

→ **Circular wait:** a cycle exists, for example P1 waits for GPU held by P2, P2 waits for licence held by P3, and P3 waits for storage held by P1.

Resource-allocation graph example:

GPU ──allocated to──> P1 ──requests──> Licence
Licence ─allocated to─> P2 ──requests──> Storage
Storage ─allocated to─> P3 ──requests──> GPU

Cycle: GPU → P1 → Licence → P2 → Storage → P3 → GPU

A cycle in a resource-allocation graph is necessary for deadlock. With one instance of each resource type, a cycle implies deadlock; with multiple instances, a cycle indicates the possibility of deadlock.

6B. Compare deadlock strategies

Strategy	**How it works**	**Evaluation for the research cluster**
Prevention	Break at least one necessary condition, e.g. enforce resource ordering or request all resources together.	Strong guarantee but may reduce utilization and flexibility; requiring all resources up front can leave GPUs idle.
Avoidance	Grant a request only if the resulting state remains safe; Banker’s algorithm is the classic method.	Prevents unsafe allocations but needs each job’s maximum resource demand in advance and adds request-time computation.
Detection and recovery	Allow allocations; periodically detect cycles/unsafe waits, then abort or roll back a selected job.	Flexible and often practical when maximum demands are unknown; detection and lost computation are costs.
Ignore (ostrich)	Do not handle rare deadlocks automatically; rely on manual restart.	Simple but unacceptable for a high-value cluster if deadlocks are frequent or jobs are expensive.

6C. Banker’s algorithm – completed program

The Need matrix is Need = Maximum − Allocation. The safety algorithm starts with Work = Available and repeatedly finds an unfinished process whose Need is no greater than Work. It assumes that process finishes, releases its allocation, and appends it to the safe sequence.

```
#include <stdio.h>
#include <string.h>
#define P 5
#define R 3
int avail[R]={3,3,2};
int maxm[P][R]={{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
int alloc[P][R]={{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};

int is_safe(int seq[P]) {
    int work[R], finish[P]={0}, count=0;
    memcpy(work,avail,sizeof work);
    while(count<P) {
        int found=0;
        for(int p=0;p<P;p++) {
            if(finish[p]) continue;
            int possible=1;
            for(int r=0;r<R;r++)
                if(maxm[p][r]-alloc[p][r]>work[r]) possible=0;
            if(possible) {
                for(int r=0;r<R;r++) work[r]+=alloc[p][r];
                finish[p]=1; seq[count++]=p; found=1;
            }
        }
        if(!found) return 0;
    }
```

```
    return 1;
}
void request(int p,const int req[R]) {
    for(int r=0;r<R;r++)
        if(req[r]>maxm[p][r]-alloc[p][r] || req[r]>avail[r]) {
            printf("P%d request: must wait (not available)\\n",p);
            return;
        }
    for(int r=0;r<R;r++){avail[r]-=req[r];alloc[p][r]+=req[r];}
    int seq[P];
    if(is_safe(seq)) {
        printf("P%d request: granted, safe sequence",p);
        for(int i=0;i<P;i++) printf(" P%d",seq[i]);
        printf("\\n");
    } else {
        for(int r=0;r<R;r++){avail[r]+=req[r];alloc[p][r]-=req[r];}
        printf("P%d request: denied (unsafe)\\n",p);
    }
}
int main(void) {
    int seq[P];
    if(is_safe(seq)) {
        printf("initial state safe:");
        for(int i=0;i<P;i++) printf(" P%d",seq[i]);
        printf("\\n");
    }
    request(1,(int[R]){1,0,2});
    request(4,(int[R]){3,3,0});
    request(0,(int[R]){0,2,0});
    return 0;
}
```

Compile and run: **gcc -std=c11 -Wall -Wextra banker.c -o banker**, then **./banker**.

Expected output:

```
initial state safe: P1 P3 P4 P0 P2\nP1 request: granted, safe sequence P1 P3 P4 P0 P2\nP4 request: must wait (not available)\nP0 request: denied (unsafe)
```

6C.1 Why P0’s request (0, 2, 0) is unsafe

Initially Available = (3,3,2). After the earlier P1 request (1,0,2) is granted, Available becomes (2,3,0). P0 requests (0,2,0), so the tentative Available becomes (2,1,0), and P0 allocation becomes (0,3,0).

The remaining Need values after this tentative grant include P0=(7,2,3), P1=(0,2,0), P2=(6,0,0), P3=(0,1,1), and P4=(4,3,1). With Work=(2,1,0), no process has Need ≤ Work: P1 needs two licences but only one is available; P3 needs one storage unit but none is available; the other processes need more resources. Therefore no process can finish, so the state is unsafe and the request is rolled back.

6D. Time-complexity prediction

The safety check may scan up to n processes repeatedly, and each process check compares m resource types. The common worst-case complexity is $O(m \times n^2)$, with O(n + m) auxiliary space (plus the input matrices). A request check invokes the safety check after a tentative allocation, so it has the same asymptotic order.

6E. Illustrative measurement

The following estimates illustrate expected growth; they are not actual measurements from this computer. A simple implementation with small fixed resource count and optimized C code may show values in this approximate range.

Processes n	**Resource types m**	**Illustrative safety-check time**	**Trend**
10	3	~0.5–2 μs	Very small.
100	3	~10–40 μs	More process scans.
1,000	3	~1–5 ms	Quadratic growth becomes visible.

To measure locally, place repeated calls to is_safe() inside a loop and time them with clock_gettime(CLOCK_MONOTONIC). Use generated safe matrices for each n, repeat enough iterations to exceed timer resolution, divide total time by the number of calls, and report the median of several runs. The input matrices themselves require O(nm) memory.

6F. Final recommendation and alternative detection/recovery

Recommendation: use a hybrid strategy. Apply prevention rules to avoid common circular waits (global resource ordering, timeouts and short leases); use Banker-style avoidance for a small, controlled set of jobs whose maximum GPU/licence/storage demands are known; and retain deadlock detection and recovery for workloads whose maximum demands cannot be predicted.

Banker's algorithm has $O(mn^2)$ worst-case work per safety check and requires declared maximum demands. That can be expensive for thousands of dynamic jobs and can reject requests that are safe in practice but cannot be proven safe from the supplied maximum claims. Detection and recovery has no safety-check cost on every request, but periodic graph detection consumes resources and recovery may discard expensive GPU computation.

An alternative detector can build a wait-for graph and search for cycles. If a cycle is found, the scheduler selects a victim based on checkpoint age, priority, resource usage and restart cost, then aborts or rolls back that job and releases its resources. Checkpointing reduces lost work. For a research cluster, this hybrid approach provides practical flexibility while preventing predictable deadlocks.

Overall Conclusion

Process management and deadlock handling must be designed around workload characteristics. A bounded worker pool prevents an examination server from creating thousands of expensive execution contexts. Scheduling policies such as SRTF and MLFQ improve responsiveness for short video jobs while avoiding permanent starvation through ageing and priority boosts. Threads provide efficient parallelism for camera processing, while processes provide stronger isolation. Pipes are simple for streaming sensor readings; shared memory is more suitable for high-volume transfers when synchronization is implemented correctly. Semaphores, mutexes, atomics and monitors prevent race conditions in booking and producer–consumer systems. Finally, deadlock prevention, avoidance, detection and recovery should be combined according to whether resource demands are known and how costly job loss would be.

Appendix – Commands and Evidence Checklist

→ Q1: compile and run q1_processes.c; capture ps output and one /proc//status file.

→ Q2: compile rr.c; run with quantum values 1, 3, 5 and 20; compare Gantt charts and averages.

→ Q3: compile q3_threads.c with -pthread; run worker counts 1, 4, 8 and 16.

→ Q4: compile q4_pipe.c; verify EOF behaviour; implement and benchmark shared memory if required.

→ Q5: compile q5_buffer.c with -pthread; run it 100 times and confirm the sum is 210 every time.

→ Q6: compile banker.c; verify the safe sequence and the three resource-request decisions.

All timing and throughput examples in this report are estimates for explanation. If the instructor requires measured evidence, execute the programs on the target Linux machine and replace each illustrative value with the observed result.